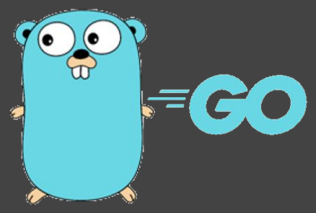

A clean and neat way for managing configuration data from environment variables

Tiago Melo
Senior software engineer

tiagoharris@gmail.com

1. Install the modules

```
$ go get github.com/kelseyhightower/envconfig
$ go get github.com/joho/godotenv
$ go get github.com/pkg/errors
```

2. Write .env file at project root dir

```
POSTGRES_USER=dbuser
POSTGRES_PASSWORD=dbpass
POSTGRES_DB=mydb
POSTGRES_HOST=localhost:5432
```

3. Write .env.local file at project root dir

```
POSTGRES_USER=localdbuser
POSTGRES_PASSWORD=localdbpass
POSTGRES_DB=localdb
POSTGRES_HOST=localhost:5433
```

4. config/config.go: struct to hold config data

```go
package config

import (
	"github.com/joho/godotenv"
	"github.com/kelseyhightower/envconfig"
	"github.com/pkg/errors"
)

// Config holds all configuration needed by this app.
type Config struct {
	PostgresUser     string `envconfig:"POSTGRES_USER" required:"true"`
	PostgresPassword string `envconfig:"POSTGRES_PASSWORD" required:"true"`
	PostgresDb       string `envconfig:"POSTGRES_DB" required:"true"`
	PostgresHost     string `envconfig:"POSTGRES_HOST" required:"true"`
}
```

5. config/config.go: reading config

```go
// Read reads the environment variables from .env file and returns a Config.
// It assumes .env file is in the current path.
func Read() (*Config, error) {
	if err := godotenv.Load(); err != nil {
		return nil, errors.Wrap(err, "loading env vars")
	}
	config := new(Config)
	if err := envconfig.Process("", config); err != nil {
		return nil, errors.Wrap(err, "processing env vars")
	}
	return config, nil
}
```

6. config/config.go: reading config from specific .env file

```go
// Read reads the environment variables from .env file at the given path
// and returns a Config.
func ReadFromFile(envFile string) (*Config, error) {
	if err := godotenv.Load(envFile); err != nil {
		return nil, errors.Wrap(err, "loading env vars")
	}
	config := new(Config)
	if err := envconfig.Process("", config); err != nil {
		return nil, errors.Wrap(err, "processing env vars")
	}
	return config, nil
}
```

7. cmd/main.go: reading config

```go
package main

import (
	"fmt"
	"os"

	"tiago.com/config"
)

func main() {
	// reading from .env file
	cfg, err := config.Read()
	if err != nil {
		fmt.Println("error when reading config:", err)
		os.Exit(1)
	}
	fmt.Printf("cfg: %+v\n", cfg)

	// reading from .env.local file
	localCfg, err := config.ReadFromFile(".env.local")
	if err != nil {
		fmt.Println("error when reading config:", err)
		os.Exit(1)
	}
	fmt.Printf("localCfg: %+v\n", localCfg)
}
```

8. cmd/main.go: running it

```
$ go run cmd/main.go

cfg: &{PostgresUser:dbuser
PostgresPassword:dbpass PostgresDb:mydb
PostgresHost:localhost:5432}

localCfg: &{PostgresUser:dbuser
PostgresPassword:dbpass PostgresDb:mydb
PostgresHost:localhost:5432}
```

Tiago Melo

github.com/tiagomelo

tiagoharris@gmail.com